SlimZero

Product Requirements Document · v1.0

Zero-overhead prompt compression, response minimisation, hallucination guarding, and autonomous agent orchestration.

Status: Draft — Open Source v0.1 Target

Stack: Python 3.10+ · MIT License · pip installable

Compatible: Anthropic · OpenAI · Gemini · Ollama · Any OpenAI-spec API

1. Executive Summary

SlimZero is a model-agnostic Python middleware library that sits between any application and any LLM API. It performs four jobs — none of which cost a single API token:

- Automatically rewrites user prompts to be shorter and clearer, reducing input tokens by 40–70%.
- Injects a minimal-response instruction fragment that pre-conditions the LLM to answer concisely, reducing output tokens by 30–50%.
- Guards against hallucinations by classifying query risk locally and validating the response against parsed intent.
- Orchestrates long-horizon autonomous agent tasks via an integrated Ralph loop and GSD task graph, with full fault prevention and circuit-breaker error control.

All compression, classification, and validation logic runs locally using lightweight models (spaCy, T5-small, sentence-transformers). The target LLM API is called exactly once — for the actual user query — and never for any meta-work.

SlimZero is designed for drop-in integration: three lines of code add it to any existing project. It ships as a PyPI package, exposes a plugin API for extensibility, and produces a real-time savings dashboard that makes cost reduction visible and shareable.

Core promise: every token spent reaches the LLM doing real work. SlimZero spends zero tokens operating itself.

2. Problem Statement

2.1 Token waste is a structural problem

Developers and users interacting with large language models such as Claude Opus, Claude Sonnet, GPT-4o, and Gemini Ultra face three compounding costs:

- Input token waste — prompts are written for human clarity, not LLM efficiency. Filler phrases, redundant instructions, verbose few-shot examples, and repeated conversation history inflate token counts with no benefit to response quality.
- Output token waste — without explicit constraints, models pad responses with preamble ("Certainly! I would be happy to…"), restated questions, and trailing summaries. These tokens are billed but discarded by most users.
- Meta-token waste — existing optimisation tools (LangChain optimisers, DSPy prompt compilation) themselves call the LLM to optimise prompts. The savings they produce are partially offset by the tokens they consume doing the optimisation.

2.2 Hallucination is an unmitigated default

Current LLM pipelines have no lightweight local layer to detect hallucination-prone queries before they reach the model, or to validate that the response actually addresses the user intent. A full RAG pipeline is heavyweight; most projects ship nothing.

2.3 Agentic loops have no built-in fault control

Autonomous agent frameworks (LangGraph, AutoGen, Ralph) can loop indefinitely, call tools with bad arguments, drift from the original task, or silently fail on sub-steps. Production deployments need circuit breakers, retry budgets, semantic drift detection, and human escalation — none of which are on by default.

2.4 Integration friction kills adoption

Most token efficiency tools require significant refactoring to adopt. They impose framework lock-in, replace the API client entirely, or require the developer to restructure their application around the tool. SlimZero solves this: the entire feature set is available behind a single wrapper call.

3. Goals and Non-Goals

3.1 Goals

- Reduce input token count by 40–70% on average without semantic loss.
- Reduce output token count by 30–50% through pre-conditioned response minimisation.
- Spend zero API tokens on compression, classification, validation, or orchestration logic.
- Detect and mitigate hallucination-prone queries locally, before the API call.
- Provide a Ralph-integrated autonomous agent loop with GSD task decomposition, retry budgets, semantic drift detection, and circuit breakers.
- Expose a pip-installable package with a three-line integration path for any existing project.
- Ship a plugin API so community contributors can add new compressors, validators, and agent strategies.
- Produce a real-time savings dashboard (CLI + JSON export) tracking tokens saved per session.

3.2 Non-Goals

- SlimZero does not replace the LLM client. It wraps any client the developer already uses.
- SlimZero does not provide model fine-tuning, training, or RLHF pipelines.
- SlimZero does not implement RAG retrieval — it provides hallucination risk scoring and uncertainty injection, not document retrieval.
- SlimZero does not guarantee zero hallucinations — it significantly reduces their probability and flags high-risk responses.
- SlimZero does not compress binary content, images, or multimodal inputs in v1.0.

4. Architecture Overview

4.1 Design philosophy

SlimZero is built on four principles:

1. Local-first. Every processing step that can run locally, does. No inference token is spent unless it reaches the user-facing LLM call.
2. Semantic safety. No rewrite is applied if it changes meaning. A cosine similarity gate (threshold: 0.92) sits between every compression step and the API call.
3. Composable pipeline. Each stage is a plugin implementing a standard interface. Stages can be added, removed, reordered, or replaced per-project without touching core code.
4. Fail-safe defaults. All fault prevention and error control mechanisms are on by default. The developer opts out, never opts in.

4.2 High-level component map

Component	Runs where	Token cost	Purpose
Intent Extractor	Local (spaCy)	0	Parse core task, entities, output format from raw prompt
Prompt Rewriter	Local (T5-small / Ollama)	0	Compress prompt to minimal imperative form
Semantic Guard	Local (sentence-transformers)	0	Reject rewrites that change meaning (sim < 0.92)
Few-Shot Ranker	Local (cosine sim)	0	Keep top-k most relevant examples, discard rest
History Compressor	Local (T5-small)	0	Summarise old turns; keep recent turns verbatim
Response Format Injector	Local (rule engine)	0	Append minimal-response instruction fragment
Hallucination Risk Scorer	Local (classifier)	0	Classify query risk; inject uncertainty instruction if HIGH
Token Budget Enforcer	Local (tiktoken)	0	Hard-cap total prompt tokens; trim by priority
LLM API Call	Remote API	ONLY HERE	Single call to target model with optimised payload
Response Validator	Local (sentence-transformers)	0	Verify response addresses parsed intent; flag if not

Component	Runs where	Token cost	Purpose
Hallucination Flag	Local (heuristic rules)	0	Detect confident-assertion patterns on uncertain topics
Savings Logger	Local (file/memory)	0	Record token delta; update cumulative dashboard
Ralph Agent Loop	Local orchestrator	0*	Autonomous task loop with circuit breakers (*agent steps cost tokens)
GSD Task Graph	Local (networkx)	0	Decompose goals into checkpointed sub-tasks

* Ralph agent loop steps make LLM calls as needed for agent reasoning. SlimZero applies the full compression pipeline to every agent call, minimising per-step token cost.

5. Pipeline Specification

5.1 Stage 1 — Intent Extractor

Purpose

Parse the user's raw prompt locally to extract structured intent before any rewriting occurs. This structured representation is used by downstream stages and by the response validator.

Implementation

- Library: spaCy (en_core_web_sm model, ~12 MB, CPU-only)
- Extracts: core_task (verb phrase), entities (named entities + noun chunks), output_format (inferred from keywords: "list", "table", "code", "explain"), constraints (negation clauses, conditional phrases)
- Output: IntentSchema dataclass — serialisable, hashable, diffable

Failure mode

- If spaCy is unavailable or parse fails: IntentSchema populated with full raw text as core_task. Pipeline continues — no stage may block the request entirely.

5.2 Stage 2 — Prompt Rewriter

Purpose

Convert the raw prompt into a minimal imperative form that preserves full meaning while reducing token count. This is the highest-impact compression stage.

Implementation

- Primary rewriter: T5-small fine-tuned on prompt compression pairs (locally, CPU/GPU). Falls back to Ollama (qwen3 1.7B or smaller) if T5 unavailable.
- Rewriting strategies applied in order: (a) strip filler phrases using a curated 300-entry phrase list; (b) merge semantically duplicate sentences; (c) convert hedged phrasing to imperative; (d) remove restated context already present in history.
- Hard constraint: rewritten prompt must be ≤ 85% of original token count. If the rewriter produces something longer, the original is used.

Failure mode — semantic guard (see Stage 3)

Stage 2 never writes to the API payload directly. Stage 3 validates every rewrite before it is accepted.

5.3 Stage 3 — Semantic Guard

Purpose

This is SlimZero's primary safety mechanism. No compressed prompt reaches the LLM unless it passes a semantic similarity check against the original.

Implementation

- Library: sentence-transformers (all-MiniLM-L6-v2, ~80 MB)
- Computes cosine similarity between original and rewritten prompt embeddings.
- Default threshold: 0.92. Configurable per-project (SLIMZERO_SIM_THRESHOLD env var or config dict).
- If similarity ≥ threshold: rewritten prompt accepted, delta logged.
- If similarity < threshold: rewritten prompt rejected, original sent. Rejection logged with reason for analysis.

Fault prevention

RULE: The semantic guard is the only non-bypassable stage in the pipeline. It cannot be disabled — only its threshold can be adjusted. Minimum allowed threshold: 0.80.

5.4 Stage 4 — Few-Shot Ranker

Purpose

When the prompt includes few-shot examples, rank them by relevance to the current query and retain only the top-k, reducing example token overhead without degrading task guidance.

Implementation

- Detects few-shot blocks via pattern matching (Q:/A: pairs, numbered examples, XML-tagged examples).
- Embeds each example and the current query using the same MiniLM model from Stage 3 (no additional download).
- Retains top-k examples by cosine similarity. Default k=3, configurable.
- Preserves at least 1 example if any were present — never strips all examples.

Failure mode

- If fewer than 2 examples detected: stage is skipped, no modification made.

5.5 Stage 5 — History Compressor

Purpose

In multi-turn conversations, old turns accumulate in the context window. This stage compresses turns older than a recency window into a rolling summary, keeping recent turns verbatim.

Implementation

- Recency window: last 4 turns verbatim (configurable, min 2).
- Older turns: summarised with T5-small into a single "Prior context:" prefix block.
- Summary regenerated only when the turn count crosses a threshold — not on every call. Cached and reused.
- Summary token budget: max 20% of total prompt token budget.

Failure mode

- If summarisation fails or produces output longer than input: older turns are truncated at token limit rather than summarised. No silent loss — truncation is logged and surfaced to the developer.

5.6 Stage 6 — Response Format Injector

Purpose

Pre-condition the LLM to respond minimally, without preamble, without restating the question, and without trailing padding — before the API call, not after.

Implementation

- Appends a fixed instruction fragment to the system prompt. The fragment is chosen from a 5-entry library based on detected output_format from the Intent Extractor.
- Fragments are ≤ 12 tokens each. Examples:
  - "Answer only. No preamble." (general)
  - "Return code only. No explanation unless asked." (code queries)
  - "List items only. No intro sentence." (list queries)
  - "One paragraph max. No summary." (explain queries)
- Fragment is appended, not prepended, to avoid displacing the developer's own system prompt.

Fault prevention

- If no system prompt exists in the request: one is created with only the fragment.
- If the developer's system prompt already contains explicit response-length instructions: the injector detects this and skips injection to avoid conflict.

5.7 Stage 7 — Hallucination Risk Scorer

Purpose

Classify each query by hallucination risk and conditionally inject an uncertainty instruction — locally, at zero token cost.

Risk classification

Risk tier	Query characteristics	Action taken
HIGH	Specific dates, numbers, citations, named statistics, "latest" / "current" facts	Append: "State uncertainty explicitly. Say I'm not certain if unsure."
MEDIUM	Named entities with verifiable attributes, procedural facts with versions	Append: "Verify before asserting specific details."
LOW	Creative, open-ended, opinion, general explanation queries	No action

Classifier: a 50-rule heuristic engine (regex + keyword patterns) — no model inference, no API call. Runs in < 5ms.

Failure mode

- Classifier errors default to MEDIUM tier (conservative). No query is ever treated as lower risk than the classifier can confirm.

5.8 Stage 8 — Token Budget Enforcer

Purpose

Apply a hard token ceiling to the outbound prompt. When the budget is exceeded after all other stages, trim in priority order.

Trim priority (lowest to highest — trimmed first)

5. Injected instruction fragments (keep last; re-inject at minimum size)
6. Old history summary (can be shortened further)
7. Low-ranked few-shot examples beyond top-1
8. Compressed rewrite (fall back to more aggressive compression)
9. Never trim: core task, most recent 2 turns, critical entities

Library: tiktoken (model-specific tokeniser; supports cl100k_base, o200k_base, claude tokeniser approximation).

Failure mode

- If budget cannot be met without trimming the core task: request proceeds over budget with a warning logged. The developer is never silently sent a malformed prompt.

6. Post-Processing Specification

6.1 Response Validator

Purpose

After the LLM responds, validate locally that the response actually addresses the parsed intent. Flag mismatches as potential hallucinations or off-task responses.

Implementation

- Embed the IntentSchema.core_task and the first 512 tokens of the LLM response using MiniLM.
- Compute cosine similarity. Threshold: 0.60 (intentionally lower — responses are allowed to be creative in form).
- Below threshold: response is flagged with SlimZeroWarning.OFF_TASK. The response is still returned to the user — SlimZero never suppresses LLM output.
- Flag is surfaced via: return value metadata, console warning (opt-out), and savings log entry.

6.2 Hallucination Flag

Purpose

Detect confident-sounding assertions in the response that correspond to HIGH-risk query types, flagging them for user attention.

Implementation

- Pattern library: 80 heuristic patterns targeting date assertions, numeric specifics, citation-style phrases ("according to", "as of", "the latest figure is"), and authority claims.
- Matches are highlighted in the metadata response object with character offsets.
- No second API call is made. The flag is purely local signal.

Fault prevention

- False positives are expected and acceptable — SlimZero errs toward flagging. The developer decides what to do with the flag.
- False negatives (missed hallucinations) are not claimed to be zero. SlimZero reduces probability; it does not guarantee correctness.

6.3 Savings Logger

Tracks per-call and cumulative statistics:

- original_input_tokens, sent_input_tokens, delta_input
- estimated_output_tokens (from response), savings_output_estimate

- stages_applied (list of which stages fired)
- semantic_similarity (of rewrite, if applied)
- hallucination_risk_tier, response_validated (bool), flags_raised
- cumulative_tokens_saved, cumulative_estimated_cost_usd (configurable model pricing)

Exportable: JSON per session, Markdown summary, Rich CLI dashboard (live, updates per call).

7. Ralph Autonomous Agent Loop Integration

7.1 What Ralph is

Ralph (from snarktank/ralph) is a lightweight autonomous agent loop: observe → plan → act → reflect → repeat. SlimZero integrates Ralph as a built-in agent orchestrator, applying the full compression pipeline to every agent turn and adding fault prevention that Ralph lacks natively.

7.2 SlimZero enhancements to Ralph

Circuit breaker

- Each Ralph loop instance is assigned a RetryBudget: max_steps (default 20), max_retries_per_step (default 3), max_total_tokens (configurable).
- If any budget is exhausted: the loop halts, state is checkpointed, and a HumanEscalation event is raised.
- Loop can be resumed from checkpoint after human review.

Semantic drift detector

- At each Ralph step, the current plan's embedding is compared to the original goal's embedding (MiniLM, same model used in Stage 3).
- If similarity < 0.75 for 3 consecutive steps: the loop is flagged as drifted. A re-grounding step is injected — the agent is shown the original goal and asked to confirm it is still on track.
- Drift events are logged. If the agent re-drifts after re-grounding: the loop halts.

Tool-call validator

- Every tool call emitted by the agent is validated locally before execution: required arguments present, argument types correct, known tool name.
- Invalid tool calls are rejected and the agent is given a structured error message with the validation failure reason. This counts as one retry.
- A tool call that fails 3 times on the same step triggers the circuit breaker for that step.

Action auditor

- Every tool call and its result is logged to a structured audit trail: timestamp, tool_name, arguments, result_summary, tokens_used.
- Audit trail is accessible in real time via the savings dashboard and exportable as JSON.

Compression on every step

- Every observation and plan prompt passed to the LLM by Ralph is routed through SlimZero's full pipeline.
- Agent steps benefit from the same 40–70% input compression and 30–50% output compression as regular calls.

8. GSD (Get-Shit-Done) Task Graph Integration

8.1 What GSD adds

GSD is SlimZero's long-horizon task decomposition layer. It takes a high-level goal and breaks it into a directed acyclic graph of sub-tasks, each with a checkpoint. The Ralph loop executes sub-tasks in dependency order. Completed sub-tasks are never re-executed even if the session restarts.

8.2 Task graph specification

Structure

- Implemented using networkx DiGraph.
- Each node: task_id, description, status (pending / running / done / failed / skipped), dependencies, retry_count, output (stored on completion).
- Edges: dependency relationships. A task runs only when all dependencies are done.

Decomposition

- User provides a goal string. GSD calls the LLM once (compressed via SlimZero) to decompose it into sub-tasks in structured JSON.
- Decomposition is the only LLM call GSD itself makes. All subsequent orchestration is local.
- Decomposition result is validated locally: valid JSON, no circular dependencies, all dependencies resolve.

Checkpointing

- Task graph is serialised to disk after every status change.
- On restart, SlimZero loads the graph and resumes from the first pending node.
- Checkpoint format: JSON (human-readable, editable by the developer if needed).

Failure handling in GSD

Failure type	GSD response	Escalation if unresolved
Sub-task fails once	Retry (up to max_retries_per_step)	Mark failed; skip or halt per policy
Sub-task fails at max retries	Mark node as failed; evaluate dependents	Human escalation event raised
Dependency chain broken	Mark all dependents as skipped	Logged; developer notified
Graph validation fails	Reject decomposition; re-request	Halt after 3 failed decompositions

Failure type	GSD response	Escalation if unresolved
	from LLM	
Checkpoint write fails	Log error; continue in-memory	Warning raised every 5 steps

9. Fault Prevention and Error Control

Fault prevention is a first-class concern in SlimZero, not an afterthought. Every component has defined failure modes, fallbacks, and escalation paths.

9.1 Layered defence model

Layer	Mechanism	Trigger condition	Response
L1 — Input validation	Schema check on every pipeline input	Missing required fields, null prompt	Raise SlimZeroInputError with field name
L2 — Semantic gate	Cosine similarity check post-rewrite	Similarity < threshold	Reject rewrite; use original; log rejection
L3 — Budget enforcer	Token count check post-compression	Prompt exceeds budget	Trim in priority order; log trim decisions
L4 — Response gate	Intent similarity check post-call	Response similarity < 0.60	Flag OFF_TASK; return response with warning
L5 — Hallucination flag	Heuristic pattern scan on response	Confident assertion on HIGH-risk query	Annotate response metadata; log flag
L6 — Circuit breaker	Step/retry/token budget exhaustion	Any budget exceeded in agent loop	Halt loop; checkpoint; raise HumanEscalation
L7 — Drift detector	Consecutive plan embedding divergence	3 steps below 0.75 similarity	Inject re-grounding; halt if re-drifts
L8 — Tool validator	Argument schema check before tool call	Missing/wrong-type argument	Reject call; return structured error to agent

9.2 SlimZero exception hierarchy

SlimZeroError (base)

- SlimZeroInputError — invalid input to pipeline
- SlimZeroRewriteError — rewriter produced invalid output
- SlimZeroSemanticRejection — rewrite rejected by semantic guard (not an error; logged as INFO)
- SlimZeroBudgetWarning — prompt could not be brought within budget
- SlimZeroResponseWarning — response failed intent validation
- SlimZeroHallucinationFlag — hallucination heuristic triggered
- SlimZeroAgentError (base for agent errors)
    - SlimZeroCircuitBreaker — loop halted by budget exhaustion
    - SlimZeroDriftHalt — loop halted by semantic drift
    - SlimZeroToolValidationError — tool call rejected by validator

- SlimZeroHumanEscalation — requires human review to resume

9.3 Global fault prevention rules

RULE 1: SlimZero never suppresses an LLM response. Flags and warnings are metadata — the response always reaches the caller.

RULE 2: SlimZero never blocks a request. If every compression stage fails, the original unmodified prompt is sent. The pipeline degrades gracefully, never silently.

RULE 3: All errors are logged before being raised. No exception leaves SlimZero without a structured log entry (JSON) with timestamp, stage, and context.

RULE 4: The semantic guard cannot be disabled. This is the one inviolable rule. Threshold may be lowered to 0.80 minimum; the guard itself cannot be removed.

RULE 5: Agent loops always checkpoint before raising HumanEscalation. No work is lost when a loop is halted.

10. Integration API — Drop-In Usage

10.1 Installation

pip install slimzero

Optional extras:

- pip install slimzero[agent] — includes Ralph loop and GSD task graph
- pip install slimzero[dashboard] — includes Rich live dashboard
- pip install slimzero[all] — everything

10.2 Minimal integration — three lines

```
from slimzero import SlimZero

sz = SlimZero(model="claude-sonnet-4-6")

response = sz.call(prompt="Explain gradient descent in detail please.")
```

That is the entire integration. SlimZero wraps the developer's existing API client and applies the full pipeline transparently.

10.3 Full configuration reference

Parameter	Default	Description
model	"claude-sonnet-4-6"	Target model identifier (any OpenAI-spec model string)
api_client	None (auto-detect)	Pass an existing Anthropic/OpenAI client; SlimZero wraps it
token_budget	None	Hard token ceiling for outbound prompt
sim_threshold	0.92	Semantic similarity gate. Min 0.80.
few_shot_k	3	Number of few-shot examples to retain
history_window	4	Number of recent turns kept verbatim
modules	all enabled	List of module names to enable; omit to disable specific stages
hallucination_guard	True	Enable hallucination risk scoring and uncertainty injection
response_validation	True	Enable post-response intent similarity check
agent_mode	False	Enable Ralph loop and GSD integration
max_agent_steps	20	Circuit breaker: max steps per Ralph loop

Parameter	Default	Description
max_retries	3	Circuit breaker: max retries per sub-task
drift_threshold	0.75	Semantic drift detection threshold for agent loop
dashboard	False	Enable Rich live savings dashboard in terminal
log_file	None	Path for JSON structured log output

10.4 Agent mode usage

```
from slimzero import SlimZero

sz = SlimZero(model="claude-opus-4-6", agent_mode=True, max_agent_steps=30)

result = sz.run_goal(
    goal="Research the top 5 open-source vector databases and write a comparison report.",
    tools=[search_tool, write_tool, read_tool]
)
```

SlimZero decomposes the goal into a GSD task graph, runs it via the Ralph loop, applies compression on every step, enforces all circuit breakers, and returns the final result with a full audit trail.

11. Plugin API — Extending SlimZero

Every pipeline stage is a plugin. Community contributors can add new compressors, validators, hallucination heuristics, and agent strategies without modifying SlimZero's core.

11.1 Stage plugin interface

```
from slimzero.plugins import BaseStage, StageInput, StageOutput

class MyCompressor(BaseStage):
    name = "my_compressor"
    def process(self, inp: StageInput) -> StageOutput:
        # inp.prompt, inp.intent, inp.token_count available
        new_prompt = ...  # your compression logic
        return StageOutput(prompt=new_prompt, modified=True, notes="reason")
```

11.2 Registering plugins

SlimZero auto-discovers plugins via Python entry points (slimzero.stages group) or manual registration:

```
sz = SlimZero(model="...", extra_stages=[MyCompressor()])
```

11.3 Plugin contracts

- A plugin must never raise an exception that halts the pipeline — catch internally, return StageOutput(modified=False) on failure.
- A plugin must never make an external API call — local inference only.
- A plugin that modifies the prompt must set modified=True so the semantic guard re-runs after it.

12. Target Benchmarks

The following benchmarks are targets for v1.0 on a standard developer laptop (CPU-only or single GPU). Actual results will be published after benchmark suite implementation.

Metric	v1.0 Target	Measurement method
Input token reduction (avg)	40–70%	Benchmarked on ShareGPT-5K prompt dataset
Output token reduction (avg)	30–50%	Estimated from response format injection ablation
Meta tokens spent by SlimZero	0	Verified by API call log — exactly 1 call per user query
Semantic similarity (avg accepted rewrites)	> 0.94	MiniLM cosine similarity on accepted rewrites
Rewrite rejection rate	< 8%	Fraction of rewrites below 0.92 threshold
Hallucination flag precision	> 0.70	Tested on TruthfulQA subset
Local pipeline latency (CPU)	< 150ms	p95 on M1 MacBook / i5-12500H
Local pipeline latency (GPU)	< 40ms	p95 on RTX 4050 6GB
Agent loop overhead vs raw Ralph	< 5%	Step latency delta with compression enabled
pip install size (base, no extras)	< 120 MB	Including spaCy sm model

13. Repository Structure

Path	Contents
slimzero/	Main package
slimzero/core.py	SlimZero class, pipeline runner, config
slimzero/stages/	One file per pipeline stage
slimzero/stages/intent.py	Intent Extractor (spaCy)
slimzero/stages/rewriter.py	Prompt Rewriter (T5/Ollama)
slimzero/stages/semantic_guard.py	Semantic Guard (MiniLM)
slimzero/stages/few_shot.py	Few-Shot Ranker
slimzero/stages/history.py	History Compressor
slimzero/stages/injector.py	Response Format Injector
slimzero/stages/hallucination.py	Hallucination Risk Scorer
slimzero/stages/budget.py	Token Budget Enforcer
slimzero/post/	Post-processing modules
slimzero/post/validator.py	Response Validator
slimzero/post/flagger.py	Hallucination Flag
slimzero/post/logger.py	Savings Logger + dashboard
slimzero/agent/	Agent orchestration
slimzero/agent/ralph.py	Ralph loop integration + circuit breakers
slimzero/agent/gsd.py	GSD task graph (networkx)
slimzero/agent/tool_validator.py	Tool call argument validator
slimzero/agent/auditor.py	Action audit trail
slimzero/plugins/	Plugin base classes and registry
slimzero/exceptions.py	Full exception hierarchy
slimzero/schemas.py	IntentSchema, StageInput, StageOutput, SlimZeroResult
tests/	pytest + hypothesis test suite
benchmarks/	Token reduction benchmark suite
docs/	MkDocs documentation
examples/	Integration examples for Flask, FastAPI, LangChain, bare API
pyproject.toml	Package config, extras, entry points

14. Development Milestones

Milestone	Scope	Target
v0.1 — Core pipeline	Stages 1–8, post-processing, savings logger, PyPI release	Week 3
v0.2 — Hallucination guard	Risk scorer + response validator + flagger with TruthfulQA benchmarks	Week 5
v0.3 — Agent integration	Ralph loop + GSD task graph + circuit breakers + drift detector	Week 8
v0.4 — Dashboard + CLI	Rich live dashboard, session JSON export, CLI slimzero command	Week 10
v0.5 — Plugin API	BaseStage interface, entry-point discovery, plugin docs + 2 example plugins	Week 12
v1.0 — Stable release	Benchmark suite, full docs, examples repo, community contribution guide	Week 16

15. Open Source and GitHub Strategy

15.1 Why this gets starred

- Immediately useful — any developer paying Opus or Sonnet bills installs it the same day they find it.
- Model-agnostic — works with every major provider and local Ollama models. Maximum audience.
- Zero-friction integration — three lines of code, no refactoring, no framework lock-in.
- Visible ROI — the savings dashboard produces screenshots people share. "I cut my Claude bill by 52% with this library" is a tweet that writes itself.
- Plugin API — the community can extend it without forking, which creates organic contribution momentum.
- Strong README — benchmark table, animated terminal recording of the dashboard, quick-start GIF.

15.2 Positioning vs existing tools

Tool	Approach	Meta-token cost	Agent fault control	SlimZero advantage
LangChain optimisers	LLM call to rewrite prompt	HIGH	None built-in	Zero meta-tokens; richer fault control
DSPy	LLM-based prompt compilation	HIGH	None	No compilation step; runtime not training-time
LLMLingua	Token-level pruning (bert)	Low	None	Semantic guard; agent integration; GSD
Ralph (raw)	Agent loop only	None	Minimal	Adds compression + circuit breakers + GSD to Ralph
SlimZero	Local pre/post processor	ZERO	Full	The only tool combining all four capabilities

16. Appendix — Quick Reference

A. Full tech stack

Library	Version	Purpose	Extras flag
spaCy	≥ 3.7	Intent extraction (en_core_web_sm)	base

Library	Version	Purpose	Extras flag
sentence-transformers	≥ 2.7	Semantic guard + response validation	base
tiktoken	≥ 0.7	Token counting (all major models)	base
transformers + T5	≥ 4.40	Local prompt rewriter	base
Ollama Python client	≥ 0.2	Fallback local rewriter via Ollama	base
networkx	≥ 3.3	GSD task graph	agent
Rich	≥ 13.7	Live dashboard + pretty logging	dashboard
pytest + hypothesis	≥ 8.0	Testing	dev
pyproject.toml	PEP 517	Package metadata and extras	—

B. Glossary

Term	Definition
Meta-token	An API token consumed by SlimZero's own operation rather than the user's query
Semantic guard	The cosine similarity check that rejects rewrites changing prompt meaning
Circuit breaker	A hard limit on agent loop steps, retries, or token usage that halts the loop
Semantic drift	Divergence between the agent's current plan and its original goal, detected via embedding similarity
GSD	Get-Shit-Done — SlimZero's task graph module for long-horizon goal decomposition
Ralph	The autonomous observe-plan-act-reflect agent loop integrated into SlimZero's agent mode
Intent schema	Structured representation of a prompt's core task, entities, format, and constraints
Response Format Injector	The stage that appends a minimal-response instruction fragment to the system prompt

— End of SlimZero PRD v1.0 —